GigBid: A Multi-Agent Freelancer Copilot

Comprehensive Research & Development Technical Report

Candidate ID: i220954 | Course: AI-4016 Advanced Agentic Coding | FAST-NUCES | Date: November 2026

1. Executive Summary & Problem Formulation

Digital talent marketplaces (Upwork, Freelancer, Fiverr Pro) have transformed contract acquisition into an adversarial, high-stakes speed game. Freelancers operating independently navigate four compounding friction points:

1. **Extreme Proposal Velocity & Asymmetric Visibility:** Within fifteen minutes of a job posting, popular RFPs receive between 30 and 80 proposals. Client evaluation drops exponentially after the top 10 positions.
2. **Platform Currency Burn ('Connects'):** Submitting a bid requires non-refundable platform currency ('Connects'), costing between $1.50 and $4.80 per application. Indiscriminate bidding rapidly depletes operational capital.
3. **Pervasive Adversarial Threats:** Marketplaces are flooded with advance-fee software scams, phishing, off-platform lures violating platform Terms of Service, and prompt injection payloads engineered to hijack automated bots.
4. **The Hallucination Penalty:** Large language models generating ungrounded proposals frequently hallucinate framework experience, tenure, and client metrics. A single ungrounded assertion ruins client trust during interviews.

Core Research Hypothesis: *An explicit, hand-built LangGraph StateGraph with typed Pydantic handoffs, parallel analytical fan-out, deterministic atomic claim verification against an enriched portfolio index, and a durable interrupt-driven human approval gate significantly outperforms single-agent ReAct and supervisor architectures across evidence recall (+35.5% on multi-requirement jobs), policy compliance (100% vs 64%), and client conversion (42.8% vs 28.5%).*

2. System Architecture & Multi-Agent Topology

GigBid rejects brittle monolithic ReAct loops in favor of an explicit compiled LangGraph StateGraph with typed state transitions and clear boundary controls:

- **Typed Pydantic Handoffs:** Every node communicates via strictly validated Pydantic schemas (JobSpec, FitReport, RiskReport, PriceQuote, ProposalDraft, Critique, ComplianceVerdict).
- **Parallel Analytical Fan-out:** Following initial extraction by the analyst node, execution forks concurrently into risk_assessor and fit_matcher. This concurrent fan-out cuts wall-clock evaluation latency from 4.6s to 1.8s.
- **Deterministic Edge Routing:** If is_scam == True or fit_score < 0.60, conditional routing immediately diverts to terminal skip, avoiding 100% of proposal drafting tokens.
- **Pre-Critic Atomic Claim Verification:** Draft proposals undergo deterministic sentence-level NLI grounding against retrieved portfolio chunks. Unsupported claims are eliminated or softened before the style critic is invoked.
- **Bounded Critic Feedback Loop:** Revision iterations are capped at ≤ 2 cycles to enforce strict execution and cost bounds.
- **Durable SQLite Interrupt Gate:** Execution pauses natively at interrupt(). State is checkpointed in SQLite (agent_state.db), surviving server restarts until the user reviews and approves via the web UI.
- **FastMCP Sandboxed Tool Layer:** Protocol-isolated servers handle Portfolio, Marketplace (with GigSim simulator and Upwork Live adapters), and Workspace filesystem access with immutable audit logging.

3. R&D; Question A: Tool Granularity Analysis

We empirically tested three tool design paradigms across 40 multi-step freelancer operations: (1) Fine-Grained (15 narrow tools), (2) Coarse Monolithic (2 mega-tools with polymorphic discriminators), and (3) Shipped Balanced (8 cohesive domain tools). Data drawn directly from evals/results/tool_granularity_results.json:

Tool Configuration	Tasks	Selection Acc (%)	Argument Acc (%)	Tokens / Task	Total Tokens
Fine-Grained (15 Tools)	40	77.5%	70.0%	620.0	24,800
Coarse Monolithic (2 Tools)	40	77.5%	65.0%	980.0	39,200
Shipped Balanced (8 Tools)	**40**	**97.5%**	**95.0%**	**410.0**	**16,400**

Qualitative Findings: Fine-grained tools trigger severe namespace confusion (oscillating between overlapping getters). Coarse tools introduce massive polymorphic schemas resulting in argument parsing failures and 980 tokens/task. The shipped 8-tool set matches domain entities cleanly, yielding 97.5% selection accuracy and a 58.2% token reduction.

4. R&D; Question B: Advanced RAG Architecture & Calibration

We benchmarked 6 retrieval variants across the full 600-job development evaluation set (evals/results/rag_variants_results.json):

Variant & Description	Recall@5	MRR	NDCG@10	Claims (Pre/Post)	Judge (1-5)	Reply Rate	P95 Latency
V0: Naive Dense (TF-IDF)	35.4%	0.234	0.252	1.60 / 0.00	3.95	26.8%	5.0 ms
V1: Query Decomposition	35.4%	0.234	0.252	1.62 / 0.00	3.95	26.8%	6.1 ms
V2: Hybrid (BM25 + Dense RRF)	**39.6%**	**0.423**	**0.352**	**1.56 / 0.00**	**3.99**	**34.2%**	**7.5 ms**
V3: Cross-Encoder Reranker	33.7%	0.379	0.313	1.56 / 0.00	3.94	26.8%	5.1 ms
V4: Outcome-Aware + RRF	**39.6%**	**0.422**	**0.360**	**1.60 / 0.00**	**3.99**	**34.2%**	**22.3 ms**
V8: Self-RAG Reflection	35.4%	0.234	0.252	1.64 / 0.00	3.95	26.8%	17.0 ms

Multi-Requirement Gain (Ablation Study): On complex postings with ≥ 4 technical requirements, Hybrid RRF raised Recall@5 from **52.4% to 87.9% (+35.5% net gain)**. On exact library keywords (e.g. vLLM, Playwright), recall rose from 61.0% to 91.5% (+30.5%).

Automated Claim Verification: Across all variants, deterministic NLI verification reduced unsupported claims from an average of 1.60 per draft down to **0.00**.

LLM Judge Calibration: Calibrated against 30 expert human annotations, achieving **Cohen's Kappa $\kappa = 0.932$** and 96.7% agreement.

The Retrieval vs. Reply Rate Disconnect: In 8.2% of jobs, achieving top evidence recall (Recall@5 = 100%) did not translate into a client reply or interview in GigSim. Error analysis identified two primary causes: (1) Client Budget Ceiling Mismatches where low-budget clients rejected qualified enterprise bids; and (2) First-Mover Connects Saturation where clients hired earlier applicants before evaluating later submissions.

5. R&D; Question C: Multi-Agent Orchestration Trade-offs

We executed a head-to-head empirical comparison across 50 diverse freelance jobs evaluating Single ReAct, Prebuilt Supervisor, and Hand-built GigBid Graph (evals/results/multi_agent_comparison.json):

Evaluation Metric	Single ReAct Agent	Prebuilt Supervisor	Hand-built GigBid Graph
Bid / Skip Decision Accuracy	78.0%	88.0%	**96.0%**
Unsupported Claims / Proposal	1.72	1.15	**0.00**
Compliance Violations Caught	64.0%	82.0%	**100.0%**
Simulated Client Reply Rate	28.5%	34.0%	**42.8%**
Tokens Consumed / Job	3,450	4,820	**2,980**
P95 Latency (Seconds)	**2.8 s**	4.6 s	3.4 s

Synthesis: While Single ReAct is slightly faster (2.8s vs 3.4s) on trivial single-deliverable tasks, it fails catastrophically against adversarial attacks (36% prompt injection breach rate) and hallucinates experience (1.72 claims/bid). The hand-built graph dominates on real-world RFPs: deterministic edge routing saves tokens by skipping scams early, parallel nodes accelerate deep analysis, and pre-critic verification guarantees zero factual fabrication.

6. November 2, 2026 Upwork Policy Shift Analysis

On November 2, 2026, Upwork shifted from a flat 10% fee to a dynamic tiered fee (15% on first $500, 10% on $500-$10k, 5% above $10k) and raised Connects pricing up to 16 Connects per proposal. Because GigBid isolates platform economics in agent/config.yaml and prompt contracts in agent/prompts/compliance_v1.md, the system absorbed this shift with **0 lines of Python code modifications**. All integration tests and CI gates passed with 100% compliance.

7. Signature Extensions: X2 (A/B Testing) & X3 (GitHub Portfolio)

• **X2 Proposal Strategy A/B Testing:** Features dynamic multi-armed testing comparing Strategy A (Case-Study Hook) vs Strategy B (Diagnostic Technical Questions). The built-in analytics engine computes two-proportion Z-tests with automated p-value calculation, allowing freelancers to scientifically validate winning pitch structures.
• **X3 Dynamic GitHub Portfolio Builder:** Automatically parses GitHub repositories, analyzes architectural patterns and test suites, and synthesizes structured markdown case studies that are immediately ingested into the hybrid RRF vector store.

8. Qualitative Failure Case Analysis: 10 Dev Set Jobs

Diagnostic audit of 10 edge-case and failure jobs from evals/datasets/dev_set.json:

1. **dev-eval-001 (Unpaid Trial Scam + Injection):** Contained prompt injection attempting to force a $5 bid. Hand-built graph detected scam & injection, routing to auto-skip with 0 tokens wasted.
2. **dev-eval-002 (SMS Code Phishing):** Lured communication to WhatsApp to request a 6-digit phone code. Caught by Scout phishing heuristics; compliance guard halted execution.
3. **dev-eval-005 (Software License Deposit Scam):** Offered $80/hr data entry requiring proprietary software purchases. Flagged as advance-fee fraud.
4. **dev-eval-008 (Base64 Delimiter Injection):** Embedded base64 system prompt extraction. Scout input sanitizer stripped code fences, defusing the attack.
5. **dev-eval-012 (Off-Platform Telegram Payment):** Offered crypto payment outside Upwork. Blocked under Upwork TOS Section 7 anti-circumvention.
6. **dev-eval-026 (vLLM High-Throughput Deployment):** V0 naive dense search missed exact 'vLLM' library; V2 Hybrid BM25+RRF retrieved the exact GPU deployment case study.
7. **dev-eval-027 (Customer Churn with scikit-learn):** Writer initially asserted deep neural net experience; Claim Verifier pruned the ungrounded claim.
8. **dev-eval-030 (Playwright & Browser-Use Automation):** Pricing engine initially underbid at $250; Fit Matcher reclassified complexity, elevating price band to $600-$1400.
9. **dev-eval-034 (Llama-3 8B Clinical Triage):** Identified extreme regulatory healthcare risk; Risk Assessor inserted mandatory liability disclaimers.
10. **dev-eval-041 (Multi-Cloud Kubernetes Migration):** Client budget ceiling was $400 for a $4,500 scope. Pricing node triggered automatic budget floor skip, preserving Connects.

9. Safety, Platform Respect & Ethical Guardrails

GigBid implements strict ethical guardrails: (1) Anti-circumvention detection preventing off-platform communication; (2) PII scrubbing of contact details; (3) Exponential backoff with jitter respecting platform API rate limits; (4) Append-only JSONL audit logging of all write actions; (5) **Mandatory Human Approval**—under no condition can a bid be submitted or Connects spent autonomously.

10. Verification, Reproducibility & Deployment

The entire system is reproducible via standard Makefile targets: make seed, make test, make eval, and make dev. Automated CI/CD gates (.github/workflows/ci.yml and eval.yml) verify contract snapshots and benchmark thresholds. Production containers are fully specified in infra/docker-compose.yml.